JSP Example for MVC based web application

Problem Statement

Develop a web application using MVC approach which provides below two features:

- Register a customer
- Search a customer

Below details of customer have to be captured and search should be possible on customer name.

- Name
- Age
- Country
- Address
- Gender
- Marital status

Use Java POJOs as model, Servlet as controller and JSP for developing view component.

JSP Example for MVC based web application

Solution:

Generic Feature: Let us create a home page "home.jsp" which will link Create and Search Customer pages.

Step 1: Develop the home page as "home.jsp" as shown below

home.jsp

```
1<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2    pageEncoding="ISO-8859-1"%>
3<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
4<html>
5<head>
6<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
7<title>Customer Management</title>
8</head>
9<body>
10Welcome !!!
11<br></br>
12Please click following link as per the operation to be performed.
13<br></br>
14<a href="createCustomer.jsp">Create Customer</a>
15<br></br>
16<a href="customerSearch.jsp">Search Customer</a>
17</body>
18</html>
```

Fig 1: home.jsp

Step 2: Add the home page as "home.jsp" in the welcome-list of web.xml

web.xml

```
1<?xml version="1.0" encoding="UTF-8"?>
2<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns="http://java.sun.com/xml/ns/javaee"
3xmlns:web="http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd"
4xsi:schemaLocation="http://java.sun.com/xml/ns/javaee http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd"
5id="WebApp_ID" version="2.5">
6  <display-name>demoPortal</display-name>
7  <welcome-file-list>
8    <welcome-file>home.jsp</welcome-file>
9  </welcome-file-list>
10  <servlet>
11    <description></description>
12    <display-name>CustomerController</display-name>
13    <servlet-name>CustomerController</servlet-name>
14    <servlet-class>com.tcs.demo.controller.CustomerController</servlet-class>
15  </servlet>
16  <servlet-mapping>
17    <servlet-name>CustomerController</servlet-name>
18    <url-pattern>/CreateCustomer</url-pattern>
19  </servlet-mapping>
20  <servlet-mapping>
21    <servlet-name>CustomerController</servlet-name>
22    <url-pattern>/CustomerController</url-pattern>
23  </servlet-mapping>
24</web-app>
```

Fig 2: web.xml

JSP Example for MVC based web application

Feature 1: Let us now develop the "create customer" feature. Develop JSP page as below.

createCustomer.jsp

```
1<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2    pageEncoding="ISO-8859-1"%>
3<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
4<html>
5<head>
6<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
7<title>Create a Customer</title>
8<script type="text/javascript" src="validation.js"></script>
9<link rel="stylesheet" href="style.css" type="text/css"></link>
10</head>
11<body>
12
13<form name="createCustomerForm" method="post" action="CreateCustomer">
14<fieldset>
15<legend>Create Customer</legend>
16<table class="tbl" border="1">
17    <caption>Create a Customer</caption>
18    <tr>
19        <td>Name:</td>
20        <td><input type="text" name="name" id="name"></input></td>
21    </tr>
22    <tr>
23        <td>Age:</td>
24        <td><input type="text" name="age" id="age"></input></td>
25    </tr>
26    <tr>
27        <td>Country:</td>
28        <td><select name="country" style="width: 100%" id="odc">
29            <option value="--Select--">--Select--</option>
30            <option value="India">India</option>
31            <option value="Australia">Australia</option>
32            <option value="UnitedKingdom">United Kingdom</option>
33        </select></td>
34    </tr>
```

JSP Example for MVC based web application

```
createCustomer.jsp
34      </tr>
35      <tr>
36          <td>Address:</td>
37          <td><textarea name="address" id="address"></textarea></td>
38      </tr>
39
40      <tr>
41          <td>Gender:</td>
42          <td><input type="radio" name="gender" value="M">Male<input
43              type="radio" name="gender" value="F">Female</td>
44      </tr>
45      <tr>
46          <td>Marital Status:</td>
47          <td><select name="maritalStatus" style="width: 100%"
48              id="maritalStatus">
49              <option value="--Select--">--Select--</option>
50              <option value="S">Single</option>
51              <option value="M">Married</option>
52              <option value="D">Divorced</option>
53          </select></td>
54      </tr>
55  </table>
56  <br></br>
57  <input type="hidden" value="createCustomer" name="action"></input> <input
58      type="submit" value="Submit"
59      onclick=
60      return validateCustomer('createCustomerForm');
61      return false;
62  ></input>
63  </fieldset>
64  </form>
65  <br></br>
66      Click <a href="home.jsp">Home</a> to go to home page.
67
68  </body>
69  </html>
```

Fig 4: createCustomer.jsp

JSP Example for MVC based web application

Step 2: Develop the controller servlet as “CustomerController.java”.

Implement doPost method as shown below to handle create and search action done by user.

```
CustomerController.java
1  package com.tcs.demo.controller;
2
3  import java.io.IOException;
14
15 /**
16  * Servlet implementation class CustomerController
17  */
18 public class CustomerController extends HttpServlet {
19     private static final long serialVersionUID = 1L;
20
21     /**
22      * @see HttpServlet#HttpServlet()
23      */
24     public CustomerController() {
25         super();
26         // TODO Auto-generated constructor stub
27     }
28
29     /**
30      * @see HttpServlet#doGet(HttpServletRequest request, HttpServletResponse response)
31      */
32     protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
33         // TODO Auto-generated method stub
34     }
35
36     /**
37      * @see HttpServlet#doPost(HttpServletRequest request, HttpServletResponse response)
38      */
39     protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
40             System.out.println("CustomerController::doPost invoked");
41
42             String action=request.getParameter("action");
43             System.out.println("User action received by controller="+action);
44
45             if("searchCustomer".equals(action)){
46
```

The class “CustomerService” will delegate the call to “CustomerDAO” class

JSP Example for MVC based web application

```
46
47                  String searchName=request.getParameter("name");
48                  //Call service layer to do customer search
49
50                  CustomerService customerService = new CustomerService();
51                  ArrayList<Customer> customerList = customerService.searchCustomer(searchName);
52
53                  //Share the matched customers details with view
54
55                  request.setAttribute("customerList",customerList);
56                  RequestDispatcher reqDispatcher = request.getRequestDispatcher("viewCustomerList.jsp");
57                  reqDispatcher.forward(request, response);
58              }
59              else if ("createCustomer".equals(action))
60              {
61                  Customer customer = new Customer();
62                  customer.setName(request.getParameter("name"));
63                  customer.setAge(Integer.parseInt(request.getParameter("age")));
64                  customer.setCountry(request.getParameter("country"));
65                  customer.setAddress(request.getParameter("address"));
66                  customer.setGender(request.getParameter("gender"));
67                  customer.setMaritalStatus(request.getParameter("maritalStatus"));
68                  CustomerService customerService = new CustomerService();
69                  int customerID = customerService.createCustomer(customer);
70
71                  // set customer ID as an attribute which will be available to the forwarded JSP
72                  request.setAttribute("customerID",customerID);
73                  RequestDispatcher reqDispatcher = request.getRequestDispatcher("createResult.jsp");
74                  reqDispatcher.forward(request, response);
75              }
76      }
77  }
78
```

Fig 6: CustomerController.java

JSP Example for MVC based web application

CustomerService.java

```java
package com.tcs.demo.service;

import java.util.ArrayList;

public class CustomerService {

    public ArrayList<Customer> searchCustomer(String name){

        //Connect DAO to search
        CustomerDAO customerDAO = new CustomerDAO();
        ArrayList<Customer> customerList = customerDAO.searchCustomer(name);
        return customerList;

    }

    public int createCustomer (Customer customer)
    {
        //Connect DAO to create
        CustomerDAO customerDAO = new CustomerDAO();
        return customerDAO.createCustomer(customer);

    }

}
```

Fig 7: CustomerService.java

CustomerDAO.java

```java
package com.tcs.demo.dao;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import com.tcs.demo.model.Customer;
import com.tcs.demo.util.DBUtil;

public class CustomerDAO {
```

JSP Example for MVC based web application

CustomerDAO.java

```java
11
12  public int createCustomer(Customer customer) {
13      // execute search query using prepared statement
14
15      Connection conn = DBUtil.getDBConnection();
16      PreparedStatement createCustomerPS = null;
17      PreparedStatement seqPS = null;
18      ResultSet rs = null;
19      int custId = 0;
20
21      String createCustomerQuery = "INSERT INTO tbl_customer (customer_id,customer_name,age,country,address,gender,married) " +
22              "values(customer_sequence.nextval,?,?,?,?,?,?)";
23      try {
24          createCustomerPS = conn.prepareStatement(createCustomerQuery);
25          createCustomerPS.setString(1, customer.getName());
26          createCustomerPS.setInt(2, customer.getAge());
27          createCustomerPS.setString(3, customer.getCountry());
28
29          createCustomerPS.setString(4, customer.getAddress());
30
31          createCustomerPS.setString(5, customer.getGender());
32          createCustomerPS.setString(6, customer.getMaritalStatus());
33          createCustomerPS.execute();
34
35          seqPS = conn
36                  .prepareStatement("SELECT customer_sequence.CurrVal as ID FROM DUAL");
37          rs = seqPS.executeQuery();
38          if (rs.next()) {
39              custId = rs.getInt("ID");
40          }
41
42      } catch (SQLException e) {
43          System.out
44                  .println("Exception occurred while performing create customer operation.");
45          e.printStackTrace();
46      } finally {
```

```java
47              DBUtil.closeResultSet(rs);
48              DBUtil.closeStatement(createCustomerPS);
49              DBUtil.closeStatement(seqPS);
50              DBUtil.closeConnection(conn);
51
52          }
53          return custId;
54
55      }
56  }
57
```

Fig 8: CustomerDAO.java

Step 4: Develop the JSP page, "`createResult.jsp`". This JSP will show the generated Customer ID in case of successful creation of a Customer. The customer ID is shared by the controller servlet as shown in Step2

Remember in Step2, to share the customer ID obtained from service method, controller uses below API:

- `request.setAttribute("customerID",customerID);`

Also, the controller servlet delegates the control to JSP view component with file name "`createResult.jsp`" using RequestDispatcher API as shown below:

- `RequestDispatcher reqDispatcher = request.getRequestDispatcher("createResult.jsp");`
- `reqDispatcher.forward(request, response);`

"`createResult.jsp`" uses below API to fetch the data shared by controller, using the same key, namely, "`customerID`" :

- `int customerID =Integer.parseInt(request.getAttribute("customerID").toString() );`

JSP Example for MVC based web application

```
createResult.jsp
1<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2    pageEncoding="ISO-8859-1" import="java.util.*, java.lang.*" %>
3<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
4<html>
5<head>
6<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
7<title>Customer Info</title>
8<link rel="stylesheet" href="style.css" type="text/css"></link>
9</head>
10<body>
11
12Thank you!!
13<br></br>
14<p>
15<%
16int customerID = Integer.parseInt(request.getAttribute("customerID").toString() );
17
18if(customerID >0 )
19{
20    %> Customer is created. Please note the Customer ID: <%=customerID %> .
21    Click <a href="home.jsp">Home</a> to go to home page.<%
22}
23else
24{
25    %> Customer is NOT created. Please try again!!!
26    Click <a href="home.jsp">Home</a> to go to home page.<%
27}
28%>
29</p>
30</body>
31</html>
```

Fig 9: createResult.jsp

Execution Screen shots for Create case:

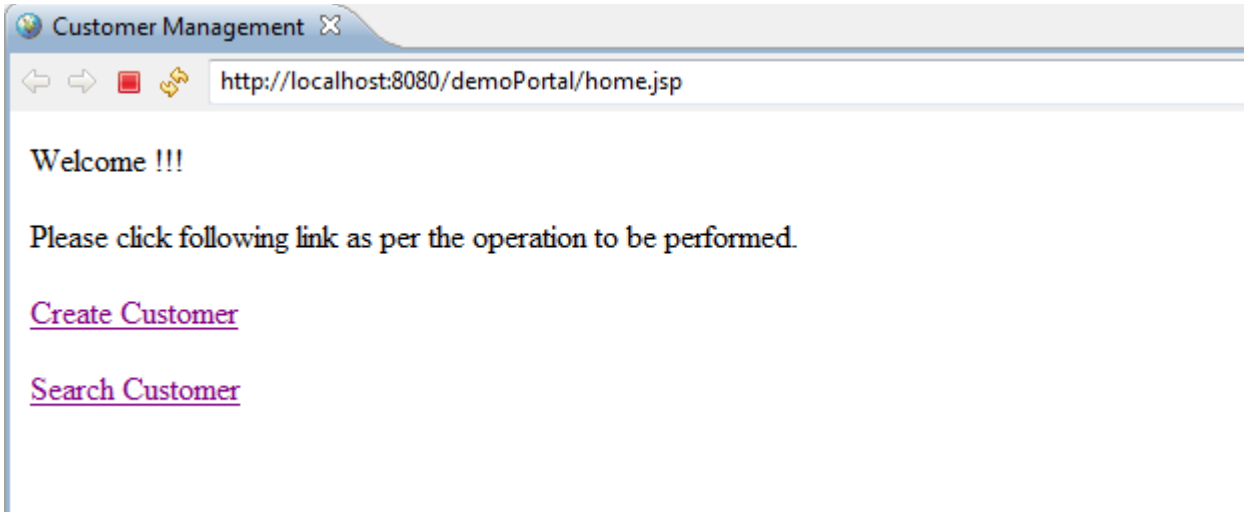


Fig 10: Home.jsp

JSP Example for MVC based web application

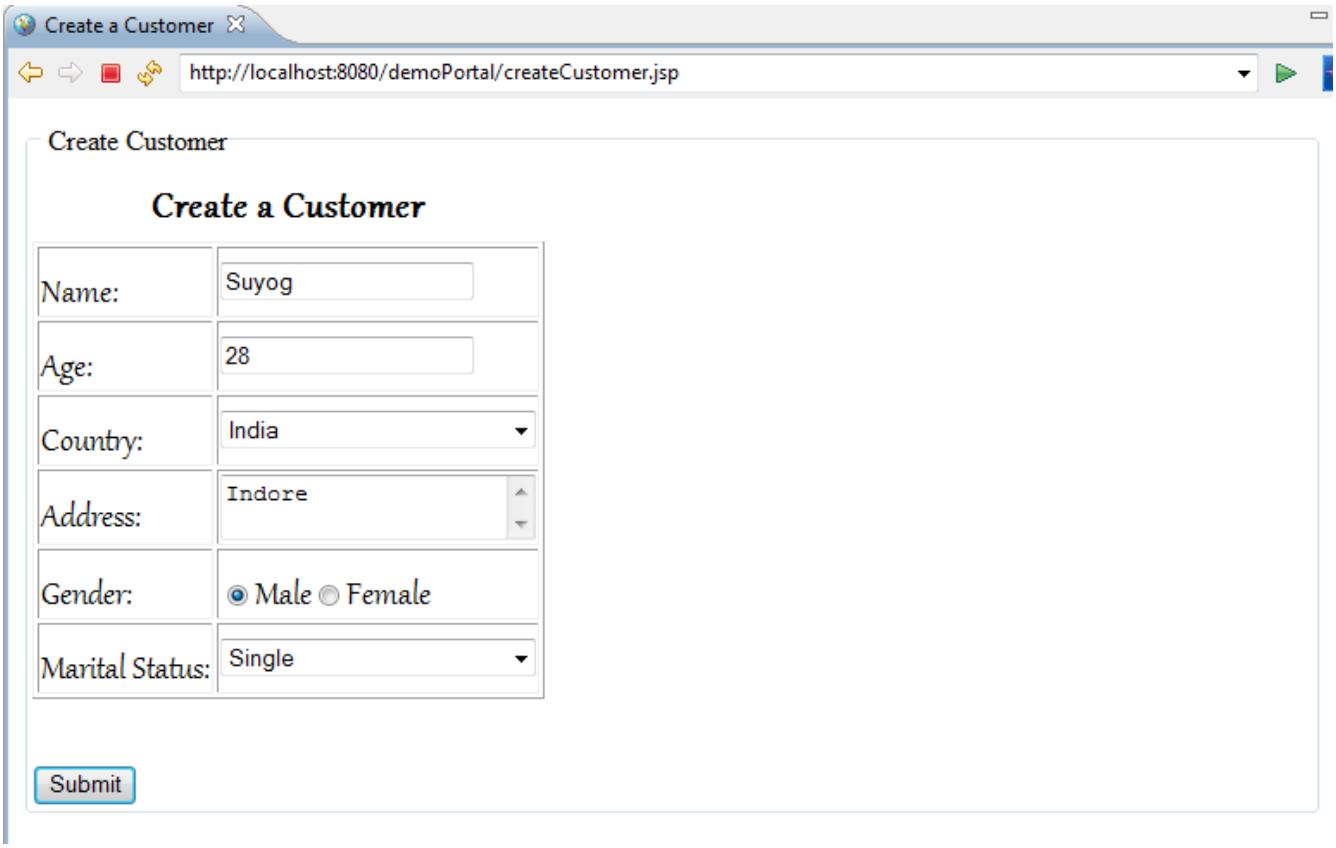


Fig 11: createCustomer.jsp

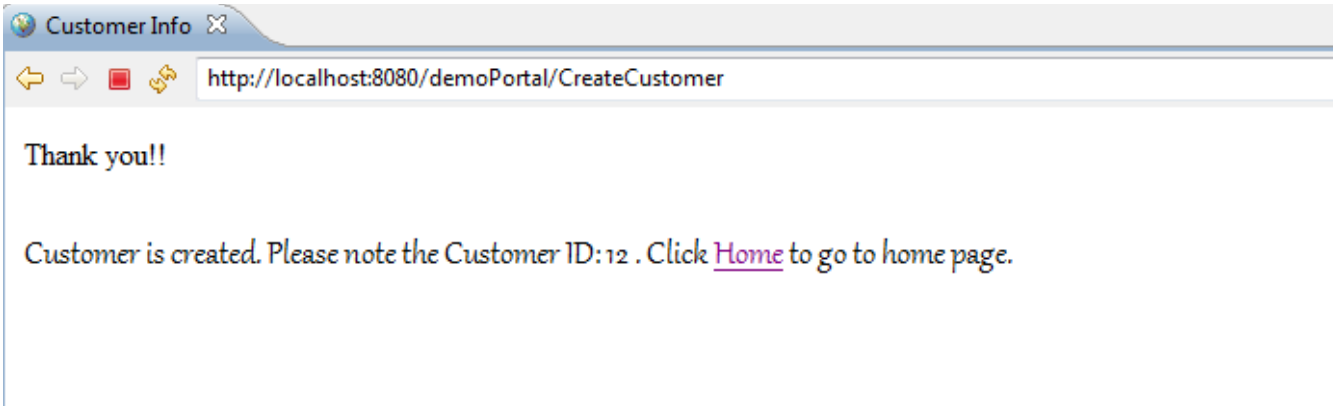


Fig 12: createResult.jsp output

JSP Example for MVC based web application

<u>Feature 2</u>: Let us now develop the "search customer" feature

Step 1: Develop the view as "customerSearch.jsp" as shown below

customerSearch.jsp

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>Insert title here</title>
</head>
<body>

<form name="customerSearch" method="POST" action="CustomerController">
    <fieldset >
        <legend>Customer Search</legend>
        <table >
            <tr><td>Customer Name</td> <td> <input type="text" name="name"  value="" /> </td> </tr>
            <tr> <td colspan="2" align="center"> <input type="submit" name="search"  value="Search" /> (Case-sensitive)</td>

        </table>

        <input type="hidden" name="action" value="searchCustomer" />
    </fieldset>
</form>

</body>
</html>
```

<u>Fig. customerSearch.jsp</u>

Step 2: Develop the controller servlet as "CustomerController.java".
Implement doPost method as shown below to handle search action done by user.
It will fetch search token and call the service layer class to search as shown below.
The class "CustomerService" will delegate the call to "CustomerDAO" class

JSP Example for MVC based web application

```
/**
 * @see HttpServlet#doPost(HttpServletRequest request, HttpServletResponse response)
 */
protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOExce
        System.out.println("CustomerController::doPost invoked");

        String action=request.getParameter("action");
        System.out.println("User action received by controller="+action);

        if("searchCustomer".equals(action)){

            String searchName=request.getParameter("name");
            //Call service layer to do customer search

            CustomerService customerService = new CustomerService();
            ArrayList<Customer> customerList = customerService.searchCustomer(searchName);

            //Share the matched customers details with view

            request.setAttribute("customerList",customerList);
            RequestDispatcher reqDispatcher = request.getRequestDispatcher("viewCustomerList.jsp");
            reqDispatcher.forward(request, response);
        }

}
```

Fig. CustomerController.java

```
public class CustomerService {

    public ArrayList<Customer> searchCustomer(String name){

        //Connect DAO to search
        CustomerDAO customerDAO = new CustomerDAO();
        ArrayList<Customer> customerList = customerDAO.searchCustomer(name);
        return customerList;

    }
```

Fig. CustomerService.java

Step 3: Develop the CustomeDAO class as shown below.

The method “searchCustomer” of this DAO class, implements the SQL query execution logic

```java
public ArrayList<Customer> searchCustomer(String name) {

    //initialize the list
    ArrayList<Customer> customerList = new ArrayList<Customer>();
    //execute search query using prepared statement
    Connection conn = DBUtil.getDBConnection();
    PreparedStatement searchCustomerPS = null;
    ResultSet rs = null;

    String searchCustomerSQL = "SELECT * FROM TBL_CUSTOMER WHERE CUSTOMER_NAME = ?";
    try {
            searchCustomerPS = conn.prepareStatement(searchCustomerSQL);
            searchCustomerPS.setString(1, name);
            rs = searchCustomerPS.executeQuery();

            while(rs.next()){
                int id = rs.getInt("CUSTOMER_ID");
                String custName = rs.getString("CUSTOMER_NAME");
                int age = rs.getInt("AGE");
                String country = rs.getString("COUNTRY");
                String address = rs.getString("ADDRESS");
                String gender = rs.getString("GENDER");
                String maritalStatus = rs.getString("MARRIED");

                Customer customer = new Customer();
                customer.setId(id);
                customer.setName(custName);
                customer.setAge(age);
                customer.setCountry(country);
                customer.setAddress(address);
                customer.setGender(gender);
                customer.setMaritalStatus(maritalStatus);

                customerList.add(customer);
            }
    } catch (SQLException e) {
        e.printStackTrace();
    }finally{
        DBUtil.closeResultSet(rs);
        DBUtil.closeStatement(searchCustomerPS);
        DBUtil.closeConnection(conn);
    }
    System.out.println("Number of customer matched="+customerList.size());
    return customerList;
}
```

Fig. CustomerDAO.java

JSP Example for MVC based web application

Step 4: Develop the JSP page, “viewCustomerList.jsp”. This JSP will iterate over the customers list shared by the controller servlet as shown in Step2

Remember in Step2, to share the customers list obtained from service method, controller uses below API:

- request.setAttribute("customerList",customerList);

Also, the controller servlet delegates the control to JSP view component with file name "viewCustomerList.jsp" using RequestDispatcher API as shown below:

- RequestDispatcher reqDispatcher = request.getRequestDispatcher("viewCustomerList.jsp");
- reqDispatcher.forward(request, response);

viewCustomerList.jsp uses below API to fetch the data shared by controller, using the same key, namely, “customerList” :

- ArrayList<Customer> customerList = (ArrayList<Customer>) request.getAttribute("customerList");

viewCustomerList.jsp

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<%@ page import="java.util.ArrayList,com.tcs.demo.model.Customer" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>Insert title here</title>
</head>
<body>
<%
    //get the customer list from request scope, that was set by controller servlet
    //casting is required, as getAttribute returns Object type
    ArrayList<Customer> customerList = (ArrayList<Customer>) request.getAttribute("customerList");

    if(customerList.size()>0) {
%>
            <table border="1">
                <tr>
                    <td>Customer ID</td>
                    <td>Name</td>
                    <td>Age</td>
                    <td>Country</td>
                    <td>Address</td>
                    <td>Gender</td>
                    <td>Is Married</td>
                </tr>
```

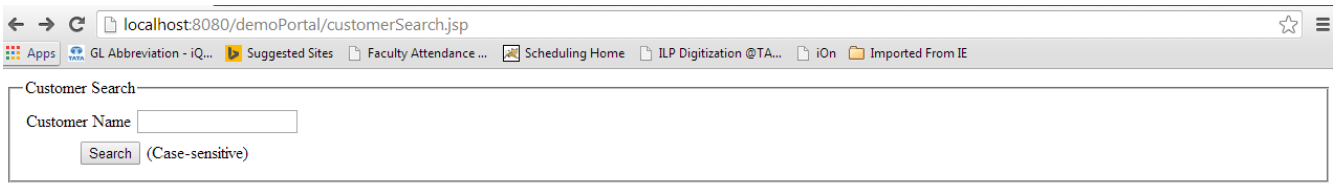
JSP Example for MVC based web application

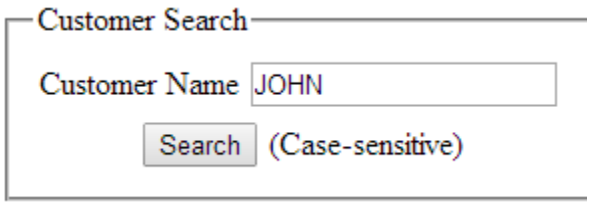
```
                <%
                    for(Customer customer:customerList){
                %>
                    <tr>
                        <td><%=customer.getId()%></td>
                        <td><%=customer.getName()%></td>
                        <td><%=customer.getAge()%></td>
                        <td><%=customer.getCountry()%></td>
                        <td><%=customer.getAddress()%></td>
                        <td><%=customer.getGender()%></td>
                        <td><%=customer.getMaritalStatus()%></td>
                    </tr>
                <%
                    }//end for
                %>
            </table>
    <%
    }//end if
    else {
    %>
                <table>
                <tr>
                    <td>No customer matched with name = <%= request.getParameter("name") %></td>
                </tr>
                </table>

    <%
    }//end else
    %>
</body>
</html>
```

Fig. viewCustomerList.jsp

JSP Example for MVC based web application

Output of Search Customer scenario:





Customer ID	Name	Age	Country	Address	Gender	Is Married
1	JOHN	35	INDIA	25, Laker Street, Mumbai	M	S

Fig. Screen shot of output for search customer flow

JSP Example for MVC based web application

Problem Assignment :

Develop a web application using MVC approach which provides below three features:

- Register a book in inventory of a Book Store
- Search a book
- Update a book's price

Below details of book have to be captured. Search should be possible on title of the book.

- Title
- Author
- Publisher
- Price
- Publish_Date
- NoOfCopies

Use Java POJOs as model, Servlet as controller and JSP for developing view component.